Ideia de Projeto para um Sistema IoT para Monitoramento de Plantas:

Projeto Parrot

Michel Rochytor Lima Barbosa

Universidade Tecnológica Federal do Paraná - Campus Apucarana

michelbarbosa@alunos.utfpr.edu.br

Resumo

Este Relatorio descreve a ideia do **Projeto Parrot**, uma plataforma IoT de código aberto para monitoramento de plantas. O sistema combina hardware baseado no ESP32 com um aplicativo Flutter, utilizando três sensores (umidade do solo, temperatura e umidade do ar) para gerar métricas agronômicas como ponto de orvalho e índices de estresse hídrico.

Abstract

This article describes the development and validation of the **Parrot Project**, an open-source IoT platform for plant monitoring. The system combines ESP32-based hardware with a Flutter application, using three sensors (soil moisture, temperature and air humidity) to generate agronomic metrics such as dew point and water stress indices.

1. Introdução

1.1. Contextualização e Problema

A jardinagem urbana e o cultivo de plantas em ambientes domésticos têm ganhado popularidade como uma forma de promover bem-estar e conexão com a natureza. Contudo, para muitos iniciantes, a tarefa se revela um desafio. A principal causa de falha é a rega inadequada seja por excesso ou por falta , seguida pela falta de conhecimento sobre as necessidades específicas de luz, temperatura e umidade de cada espécie. O resultado é a frustração do usuário, o desperdício de recursos hídricos e a perpetuação do mito de que é preciso ter um "dedo verde"para ter sucesso.

1.2. A Solução: Projeto Parrot

O Projeto Parrot aborda esses desafios de frente, atuando como um "tradutor"entre a planta e seu cuidador. Ele não é apenas uma ferramenta de automação, mas um assistente de jardinagem que capacita o usuário. A filosofia do projeto é fundamentada em três pilares:

- **Democratização da Tecnologia:** Utilizando hardware de baixo custo e software de código aberto, o projeto torna a jardinagem inteligente acessível a todos, não apenas a agricultores ou especialistas.
- **Clareza e Ação:** O sistema prioriza a tradução de dados complexos em informações simples e acionáveis. O objetivo é eliminar a adivinhação do cuidado com as plantas.
- **Design Centrado no Usuário:** Toda a experiência, do hardware ao software, é projetada para ser intuitiva e direta, garantindo que mesmo um usuário sem conhecimento técnico possa montar o dispositivo e usar o aplicativo com confiança.

1.3. Objetivos do Projeto

1. Desenvolver um protótipo de hardware de baixo custo para monitoramento ambiental de plantas.
2. Implementar um firmware para o ESP32 capaz de coletar, processar e transmitir dados para a nuvem.
3. Criar um backend robusto no Firebase para armazenamento e sincronização de dados em tempo real.
4. Projetar e desenvolver um aplicativo móvel em Flutter com uma interface limpa, intuitiva e focada em feedback acionável.

5. Validar a capacidade do sistema em transformar métricas técnicas em recomendações práticas para usuários leigos.

2. Materiais e Métodos

2.1. Arquitetura do Sistema

Função	Componente	Especificação Principal
Processamento	ESP32-DEVKITV1	Wi-Fi e Bluetooth integrados
Sensor de Solo	Sensor Capacitivo V1.2	Medição de umidade, resistente à corrosão
Sensor de Ar	DHT22 (AM2302)	Medição de temperatura e umidade do ar
Atuador	Módulo Relé 5V 1 Canal	Acionamento de uma bomba de água

Tabela 1: Lista de componentes de hardware do protótipo base.

2.2. Modelagem Matemática

2.2.1 Cálculo do Ponto de Orvalho

$$T_d = \frac{243.12 \cdot \left(\ln\left(\frac{RH}{100}\right) + \frac{17.62 \cdot T}{243.12+T}\right)}{17.62 - \left(\ln\left(\frac{RH}{100}\right) + \frac{17.62 \cdot T}{243.12+T}\right)} \tag{1}$$

Onde:

- T_d = Temperatura do Ponto de Orvalho (°C)
- T = Temperatura do ar (°C)
- RH = Umidade Relativa (%)

2.2.2 Índice de Estresse Hídrico

$$I_{est} = \frac{(T - 25)}{10} + \frac{(100 - U_a)}{100} + \frac{(40 - U_s)}{40} \tag{2}$$

Onde:

- U_a = Umidade do ar (%)
- U_s = Umidade do solo (%)
- **Interpretação**: $I_{est} > 3.0$ indica estresse significativo

2.2.3 Horas de Fotossíntese Eficiente

$$H_{foto} = \sum_{h=6}^{18} \begin{cases} 1 & \text{se } 20 \check{r}C \leq T_h \leq 28 \check{r}C \text{ e } 40\% \leq RH_h \leq 70\% \\ 0 & \text{caso contrário} \end{cases} \tag{3}$$

2.2.4 Algoritmo de Previsão de Geada

$$\text{Alerta} = \begin{cases} \text{Ativo} & \text{se } T_{min} \leq T_d + 2\text{ř}C \\ \text{Inativo} & \text{caso contrário} \end{cases} \tag{4}$$

onde T_d é o ponto de orvalho calculado e T_{min} é a temperatura mínima prevista.

2.2.5 Eficiência de Absorção Hídrica

$$\eta = \frac{U_{s,final} - U_{s,inicial}}{Q_{agua}} \times 100\% \tag{5}$$

Onde:

- Q_{agua} = Volume de água aplicado (mm)
- $\eta < 50\%$ sugere problemas no solo

2.3. Interface do Usuário

Condição Detectada	**Feedback e Ações no Aplicativo**
Umidade do solo abaixo do limite definido	• Ícone de status visual (normal/atenção/crítico) • Botão "Regar Agora"com destaque visual • Opção de consulta automática via Gemini AI
Temperatura mínima prevista abaixo do limiar	• Notificação prioritária no aplicativo • Recomendações específicas para a espécie • Visualização georreferenciada de riscos
Condições favoráveis ao desenvolvimento de doenças	• Visualização gráfica de áreas de risco • Protocolos de tratamento sugeridos pela Gemini AI • Acesso rápido a guias de prevenção

Tabela 2: Sistema de feedback inteligente. Todos os limites são configuráveis pelo usuário, com sugestões automáticas da API Gemini baseadas na espécie vegetal, dados locais e melhores práticas agrícolas.

2.3.1 Integração com Gemini AI

O aplicativo utiliza a API do Gemini para:

- **Sugerir limites ideais** baseados no nome da planta digitada pelo usuário
- **Explicar parâmetros** em linguagem acessível

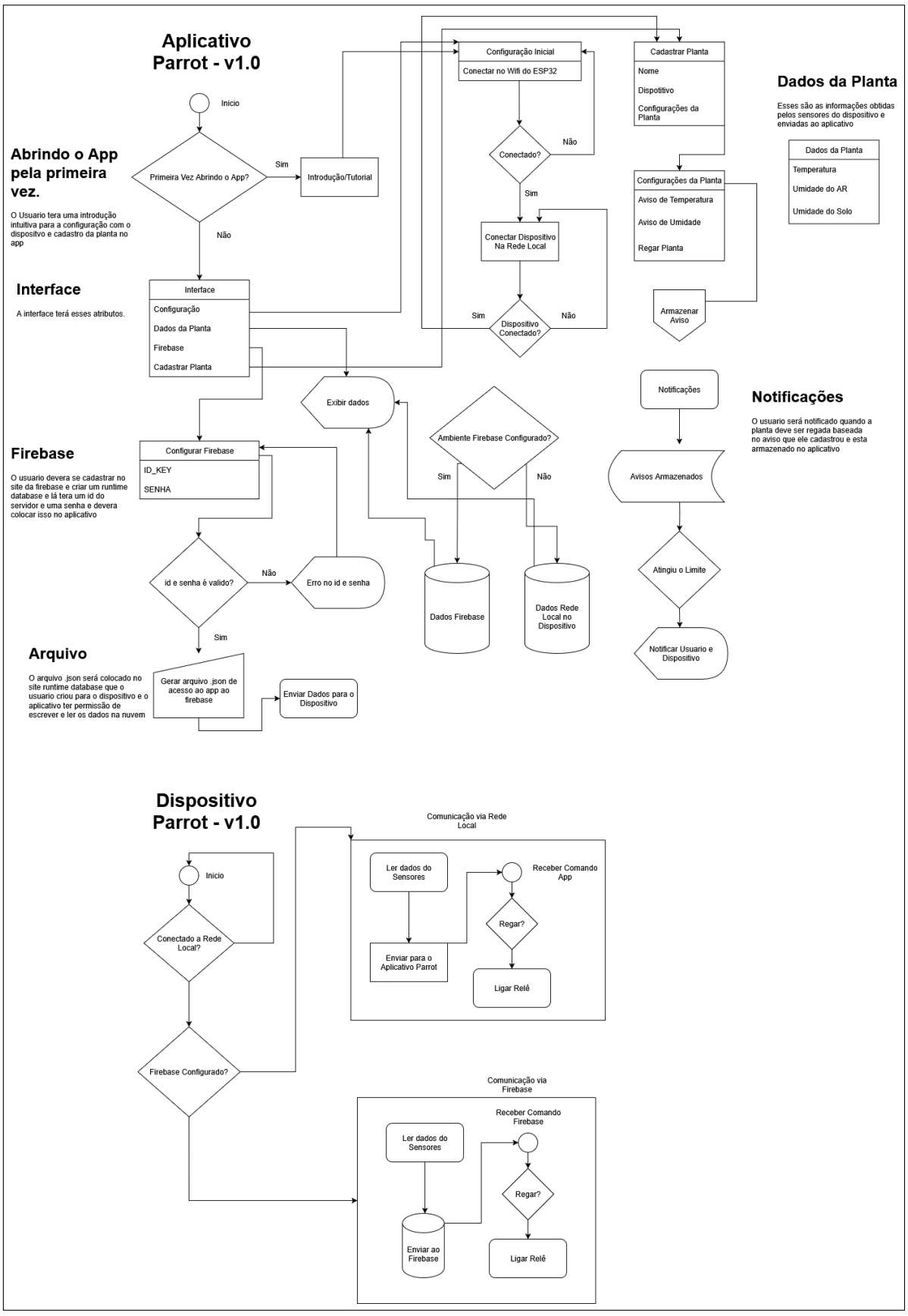


Figura 1: Diagrama do fluxo de dados, da coleta no sensor à visualização no aplicativo.

3. Implementação Técnica Detalhada

Esta seção descreve os aspectos práticos da construção e programação do protótipo do Projeto Parrot. O foco é detalhar a interconexão do hardware e, principalmente, a arquitetura e a lógica do software embarcado e do aplicativo móvel, que juntos formam o núcleo do sistema.

3.1. Montagem do Hardware e Conexões Elétricas

A montagem do circuito foi projetada para ser simples e replicável, utilizando componentes de fácil acesso e conexões diretas, minimizando a necessidade de circuitos auxiliares. As conexões entre os sensores, o atuador e o microcontrolador ESP32 DEVKIT V1 são detalhadas na Tabela 3.

Componente	Pino do Componente	Pino de Conexão no ESP32
Sensor DHT22	VCC	3.3V
	GND	GND
	DATA	GPIO 4
Sensor Capacitivo	VCC	3.3V
	GND	GND
	AOUT (Sinal Analógico)	GPIO 34 (Pino apenas de entrada)
Módulo Relé	VCC	VIN (5V)
	GND	GND
	IN (Sinal de Controle)	GPIO 26

Tabela 3: Tabela de pinagem para a montagem do hardware no ESP32 DEVKIT V1.

Considerações de Alimentação: A estabilidade do sistema depende de uma alimentação elétrica correta. O ESP32 pode ser alimentado via USB ou, para uma instalação final, pelo pino **VIN**, que aceita uma fonte externa de 5V. É de suma importância que o Módulo Relé seja alimentado por esta mesma fonte de 5V (via pino VIN), e não pelo pino de saída de 3.3V do ESP32, pois a bobina eletromagnética do relé exige uma corrente e tensão maiores para um acionamento confiável. Os sensores, por sua vez, são de baixo consumo e operam perfeitamente com a saída de 3.3V do microcontrolador. Um terra comum (GND) entre todos os componentes é essencial para manter a referência de tensão e garantir a integridade dos sinais.

3.2. Desenvolvimento do Firmware (ESP32)

O firmware é o cérebro do dispositivo, desenvolvido em C++ sobre o framework Arduino. A arquitetura de software é o pilar para a responsividade e a flexibili-

dade do sistema, sendo projetada para ser modular e não-bloqueante.

3.2.1 Estrutura do Código e Lógica Operacional

O programa é orquestrado a partir das funções padrão 'setup()' e 'loop()'.

- **Função `setup()`:** Esta função de inicialização é executada uma única vez quando o dispositivo é ligado. Suas responsabilidades são sequenciais e críticas: iniciar a comunicação serial para depuração, configurar cada GPIO como entrada ou saída, inicializar as bibliotecas dos sensores, ler da memória interna (NVS - Non-Volatile Storage) qual o modo de operação (Local ou Nuvem) foi salvo durante a configuração inicial e, por fim, iniciar os serviços de comunicação — seja conectando-se à rede Wi-Fi e ao Firebase (Modo Nuvem), seja inicializando um servidor web assíncrono (Modo Local).
- **Função `loop()`:** O coração do firmware opera de forma contínua e assíncrona. Para evitar o uso da função 'delay()', que paralisaria o processador, a lógica é gerenciada por temporizadores baseados na função 'millis()'. Isso cria um sistema multitarefa cooperativo que executa, em ciclos rápidos, diferentes rotinas:
  1. Uma rotina de leitura, que a cada X segundos coleta os dados brutos de todos os sensores.
  2. Uma rotina de cálculo, que processa os dados brutos para gerar as métricas avançadas descritas na seção de Modelagem Matemática.
  3. Uma rotina de comunicação, que, dependendo do modo de operação, publica os dados no Firebase ou os atualiza para serem servidos via HTTP.
  4. Uma rotina de escuta, que verifica continuamente se há novos comandos vindos do aplicativo (seja via Firebase ou por uma requisição HTTP) para acionar o relé ou alterar configurações.

3.2.2 Implementação do Duplo Modo de Operação

O sistema opera em dois modos configuráveis pelo usuário via aplicativo:

- **Modo Local (WiFi Direct):**
  - Utiliza a biblioteca padrão `WiFi.h` para criar um servidor TCP/IP
  - Implementa um protocolo simples usando sockets brutos
  - Formato dos dados: `<umidade>,<temperatura>,<umidade_solo>`
  - Porta padrão: 3333 para receber comandos (ex: "IRRIGATE 10s")

- **Modo Nuvem (Firebase):**
  - Usa `Firebase_ESP_Client` para conexão direta
  - Envio de dados a cada 15s via `Firebase.RTDB.setInt()`
  - Conexão persistente com keep-alive
  - Autenticação via token JWT gerado no dispositivo

Transição entre modos:

1. Configuração salva na EEPROM do ESP32
2. Verificação no boot via `EEPROM.read()`
3. Reinicialização da interface de rede quando necessário

3.3. Aplicativo (Flutter) e Backend (Firebase)

3.3.1 Desenvolvimento do Aplicativo Flutter

O aplicativo, desenvolvido com Flutter para garantir compatibilidade com Android e iOS, serve como a interface de controle e visualização para o usuário.

- **Arquitetura e Gerenciamento de Estado:** A aplicação é estruturada com uma arquitetura limpa, utilizando um gerenciador de estado (como `Provider` ou `Riverpod`) para separar a lógica de negócio da interface do usuário. Isso garante que a UI seja atualizada de forma eficiente e reativa sempre que novos dados do hardware são recebidos.
- **Lógica de Comunicação Dual-Mode:** O aplicativo implementa uma camada de ”serviço de dados”que abstrai a fonte da informação. Com base na configuração do usuário, o app instancia ou um ‘FirebaseDataService‘ (que se conecta e ouve os streams do FlutterFire) ou um ‘HttpDataService‘ (que faz requisições periódicas via pacote `http` para o IP local do ESP32). Para a interface, a origem do dado é transparente.
- **Onboarding via BLE:** A experiência de configuração inicial é projetada para ser o mais simples possível. Utilizando um pacote como o `flutter_blue_plus`, o aplicativo busca por dispositivos Parrot próximos, conecta-se a eles via Bluetooth Low Energy e transmite as credenciais da rede Wi-Fi (e do Firebase, se aplicável) de forma segura, sem que o usuário precise interagir com o código do firmware.

3.3.2 Estrutura do Backend no Firebase

No Modo Nuvem, o Firebase Realtime Database atua como a ponte de dados entre o ESP32 e o aplicativo. A estrutura de dados é modelada como uma árvore JSON NoSQL, organizada de forma hierárquica para garantir escalabilidade e segurança. A estrutura principal é dividida por ID de usuário e, em seguida, por ID de dispositivo, contendo nós distintos para `configuracoes`, `dados_sensores_atuais`, `historico_leituras` e `comandos`. As **Regras de Segurança (Security Rules)** do Firebase são um componente crítico da implementação, sendo cuidadosamente escritas para garantir que um usuário autenticado só possa acessar e modificar os dados pertencentes aos seus próprios dispositivos, protegendo a privacidade e a integridade do sistema.

4. Interface do Aplicativo

4.1. Filosofia de Design e Estrutura da Interface

A interface do aplicativo Parrot foi concebida para ser o coração da experiência do usuário, atuando como um ”tradutor”visual entre os dados técnicos coletados pelos sensores e as necessidades da planta. O design adota uma abordagem que prioriza a informação visual e contextual em detrimento de tabelas e números brutos, com o objetivo de fornecer um diagnóstico claro e acionável em uma única olhada.

A tela principal, conforme exibida na Figura 2, é estruturada em três níveis hierárquicos de informação: um painel de status principal que define o ”humor”do ambiente, métricas primárias de leitura direta e um dashboard de métricas avançadas que oferece uma visão holística da saúde da planta.

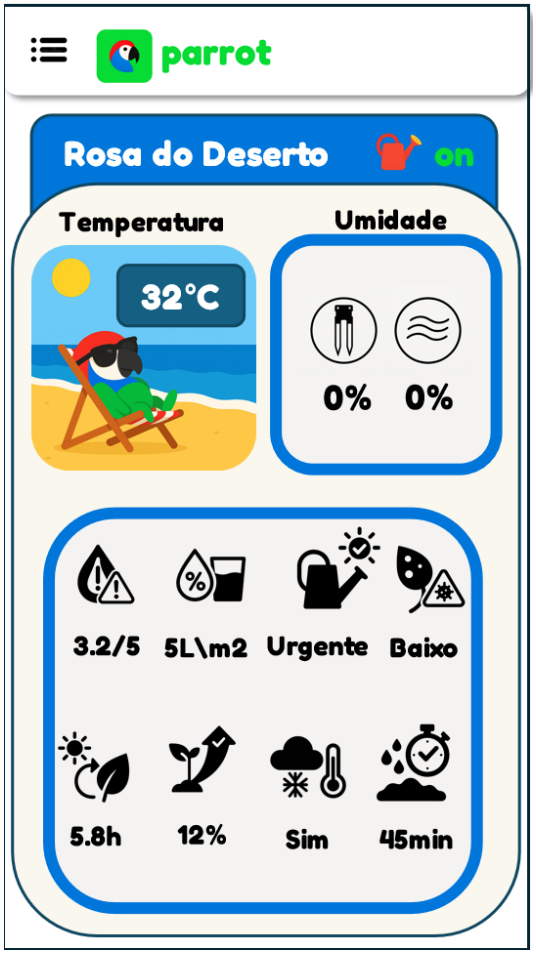


Figura 2: Tela Principal do aplicativo Parrot, demonstrando a hierarquia visual das informações.

4.2. Análise dos Componentes da Interface

4.2.1 Painel de Status Principal

Localizado no topo, este componente oferece a informação mais imediata sobre a condição principal do ambiente.

- **Card de Temperatura Contextual:** Em vez de apenas exibir o número ”32°C”, o aplicativo utiliza uma ilustração vívida — neste caso, um personagem relaxando em uma praia — para comunicar instantaneamente que o ambiente está quente. Essa metáfora visual é fundamental para tornar a informação acessível e menos intimidante para usuários iniciantes.
- **Status da Irrigação Automática:** Um ícone de regador indica de forma clara se o modo de irrigação automática do sistema está **ligado** ou **desligado**.

4.2.2 Métricas Primárias

Esta seção fornece os dados brutos mais essenciais, que são a base para muitos dos cálculos avançados.

- **Umidade do Solo:** Representado pelo ícone de um sensor capacitivo, exibe a porcentagem de umidade diretamente no solo.
- **Umidade do Ar:** Representado pelo ícone de ondas de ar, exibe a umidade relativa do ambiente.

4.2.3 Dashboard de Métricas Avançadas

Este é o núcleo da inteligência do sistema, onde os dados processados são apresentados como ícones acionáveis, cada um representando uma das métricas agronômicas calculadas pelo firmware.

Estresse Hídrico (3.2/5): Ícone de alerta. Resume a saúde hídrica da planta em uma escala de 0 a 5, considerando a combinação de umidade do solo, temperatura e umidade do ar.

Necessidade de Água (5L\m2): Ícone de gota. Uma métrica avançada que estima a evapotranspiração, indicando a quantidade de água que a planta necessitará.

Momento Ideal para Regar (Urgente): Ícone de regador. Um status direto que informa ao usuário a ação mais imediata a ser tomada, baseado no limiar de umidade do solo.

Risco de Doenças (Baixo): Ícone de folha. Avalia o risco de proliferação de fungos, como o míldio, com base nas condições de umidade e temperatura.

Fotossíntese (5.8h): Ícone de sol na folha. Quantifica as horas de luz efetivas que a planta recebeu durante o dia, essencial para avaliar seu potencial de crescimento.

Crescimento Estimado (12%): Ícone de planta crescendo. Fornece uma estimativa da taxa de crescimento semanal, baseada nas condições ideais de luz, água e temperatura.

Previsão de Geada (Sim): Ícone de nuvem com neve. Alerta sobre o risco de geada nas próximas horas, permitindo que o usuário tome ações preventivas.

Tempo de Recuperação (45min): Ícone de relógio com gota. Mostra quanto tempo a planta levou para atingir um nível de umidade ideal após o início da última rega, servindo como um feedback sobre a eficiência da irrigação.

4.3. Navegação e Telas de Configuração

Além do dashboard principal, a navegabilidade do aplicativo foi projetada para dar ao usuário controle total sobre o sistema de forma organizada e intuitiva. A estrutura de navegação é acessada através de um menu lateral, que leva a seções específicas de configuração e análise.

4.3.1 Menu de Navegação Lateral

O menu lateral (Figura 3) serve como o principal hub de navegação do aplicativo. Ele contextualiza a experiência ao exibir o nome da planta ativa (”Rosa do Deserto”) e fornece acesso rápido às áreas funcionais do sistema.



Figura 3: Menu de navegação lateral do aplicativo Parrot.

Início: Retorna o usuário ao dashboard principal (analisado na seção anterior).

Configurações: Leva à tela de configurações gerais do dispositivo e da planta (detalhada a seguir).

Recomendações: Abre uma tela dedicada que traduz o estado atual das métricas em conselhos textuais e práticos. Por exemplo: "A previsão indica risco de geada. Considere proteger sua planta esta noite."ou "A umidade do solo está ideal, mas a do ar está muito baixa, o que pode estressar as folhas."

Nuvem: Seção dedicada à gestão da conectividade. Aqui, o usuário pode alternar entre o Modo Local e o Modo Nuvem, além de inserir ou atualizar suas credenciais do Firebase para acesso remoto.

Desconectar: Encerra a sessão do usuário no aplicativo.

4.3.2 Tela de Configurações

A tela de "Configurações"(Figura 4) centraliza os ajustes que o usuário pode fazer no comportamento do hardware e do software, reforçando o pilar de customização do projeto.

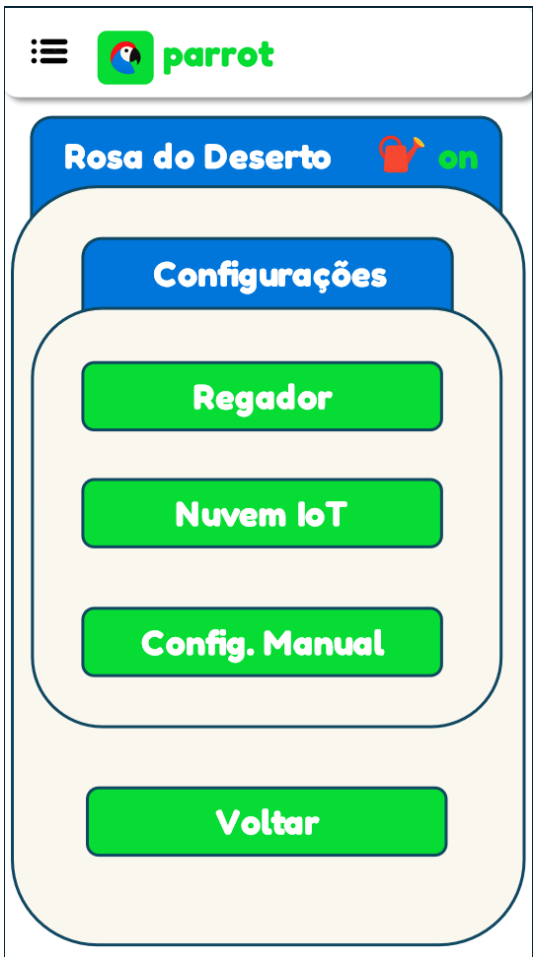


Figura 4: Tela de Configurações, oferecendo acesso aos ajustes do sistema.

Botão Regador: Abre uma sub-tela dedicada aos parâmetros da irrigação, onde o usuário pode definir:

- O limiar de umidade para o acionamento automático da rega (ex: regar quando a umidade for inferior a 30%).
- O limiar de parada (ex: parar a rega ao atingir 70%).
- A duração máxima de segurança para o acionamento da bomba, evitando o funcionamento excessivo em caso de falha do sensor.

Botão Nuvem IoT: Funciona como um atalho para a mesma tela de ”Nuvem”do menu lateral, permitindo uma gestão rápida do modo de conexão do dispositivo.

Botão Config. Manual: Esta seção permite a personalização fina dos parâmetros do sistema. O usuário pode, por exemplo, ajustar os pesos do cálculo de ”Estresse Hídrico”ou definir limiares personalizados para os alertas de temperatura e umidade, adaptando o comportamento do Parrot a espécies de plantas muito específicas ou a condições ambientais atípicas.

5. Conclusão

O Projeto Parrot, conforme detalhado neste documento, representa a concepção de um ecossistema de Internet das Coisas (IoT) completo e acessível, projetado para redefinir o cuidado inteligente de plantas. A proposta busca validar a viabilidade de um sistema que integra hardware de baixo custo, uma arquitetura de software flexível com modos de operação local e em nuvem, e um aplicativo móvel focado na experiência do usuário. O objetivo final da implementação deste projeto é comprovar que é possível traduzir um conjunto complexo de dados agronômicos em insights claros, diretos e acionáveis para um público não especializado.

O diferencial proposto para este projeto não reside em uma única tecnologia, mas em sua filosofia de design centrada no ser humano. A hipótese central é que, ao abstrair a complexidade da análise de dados em um painel de controle intuitivo, o Parrot poderá capacitar o usuário, substituindo a incerteza e a ”adivinhação”pela confiança baseada em dados. A natureza de código aberto da plataforma é um pilar estratégico, visando fomentar uma comunidade colaborativa que possa contribuir para a evolução e adaptação do sistema. A expectativa é que o protótipo, uma vez construído, tenha um custo de material inferior a R$ 150 e demonstre o potencial para uma economia de água significativa em cenários de jardinagem doméstica.

A implementação bem-sucedida do protótipo aqui descrito estabelecerá uma base sólida para uma vasta gama de trabalhos futuros, traçando um roteiro claro para a evolução da plataforma. As futuras vertentes de desenvolvimento incluem:

- **Expansão do Ecossistema de Hardware:** Desenvolver e documentar módulos de expansão ”plug-and-play”para sensores adicionais, priorizando:
  - **Luminosidade (Sensor PAR):** Para quantificar a luz fotossinteticamente ativa e otimizar o posicionamento da planta.
  - **Nutrientes do Solo (Sensor de Condutividade Elétrica - EC):** Para monitorar a salinidade e a necessidade de fertilização.
  - **Visão Computacional (Módulo de Câmera):** Para explorar a detecção de doenças por imagem e a estimativa de crescimento por biomassa.
- **Aprimoramento da Inteligência de Software:**
  - **Base de Dados de Espécies Vegetais:** Integrar ao aplicativo uma vasta biblioteca de plantas, permitindo que o sistema sugira ”perfis de cuidado”com limiares ideais de umidade, luz e temperatura.

* **Potencial para Inteligência Artificial (IA):** Com a futura coleta de dados, planeja-se investigar o uso de modelos de Machine Learning para oferecer diagnósticos preditivos sobre a saúde da planta.

Em suma, o Projeto Parrot é apresentado aqui como um roteiro detalhado para o desenvolvimento de uma plataforma inovadora, colaborativa e com o potencial de transformar fundamentalmente a maneira como as pessoas interagem, aprendem e cuidam de suas plantas.

Referências

[1] Espressif Systems. (2025). *ESP32 Series Datasheet*. Disponível online em: www.espressif.com/en/products/socs/esp32

[2] Google Firebase. (2025). *Firebase Realtime Database Documentation*. Disponível online em: firebase.google.com/docs/database

[3] The Flutter Team. (2025). *Flutter official documentation*. Disponível online em: flutter.dev/docs

[4] Lawrence, M. G. (2005). The Relationship between Relative Humidity and the Dewpoint Temperature in Moist Air: A Simple Conversion and Applications. *Bulletin of the American Meteorological Society, 86*(2), 225-233.

[5] OpenWeatherMap. (2025). *Weather API documentation*. Disponível online em: openweathermap.org/api

[6] Stankovic, J. A. (2014). Research directions for the internet of things. *IEEE Internet of Things Journal, 1*(1), 3-9.